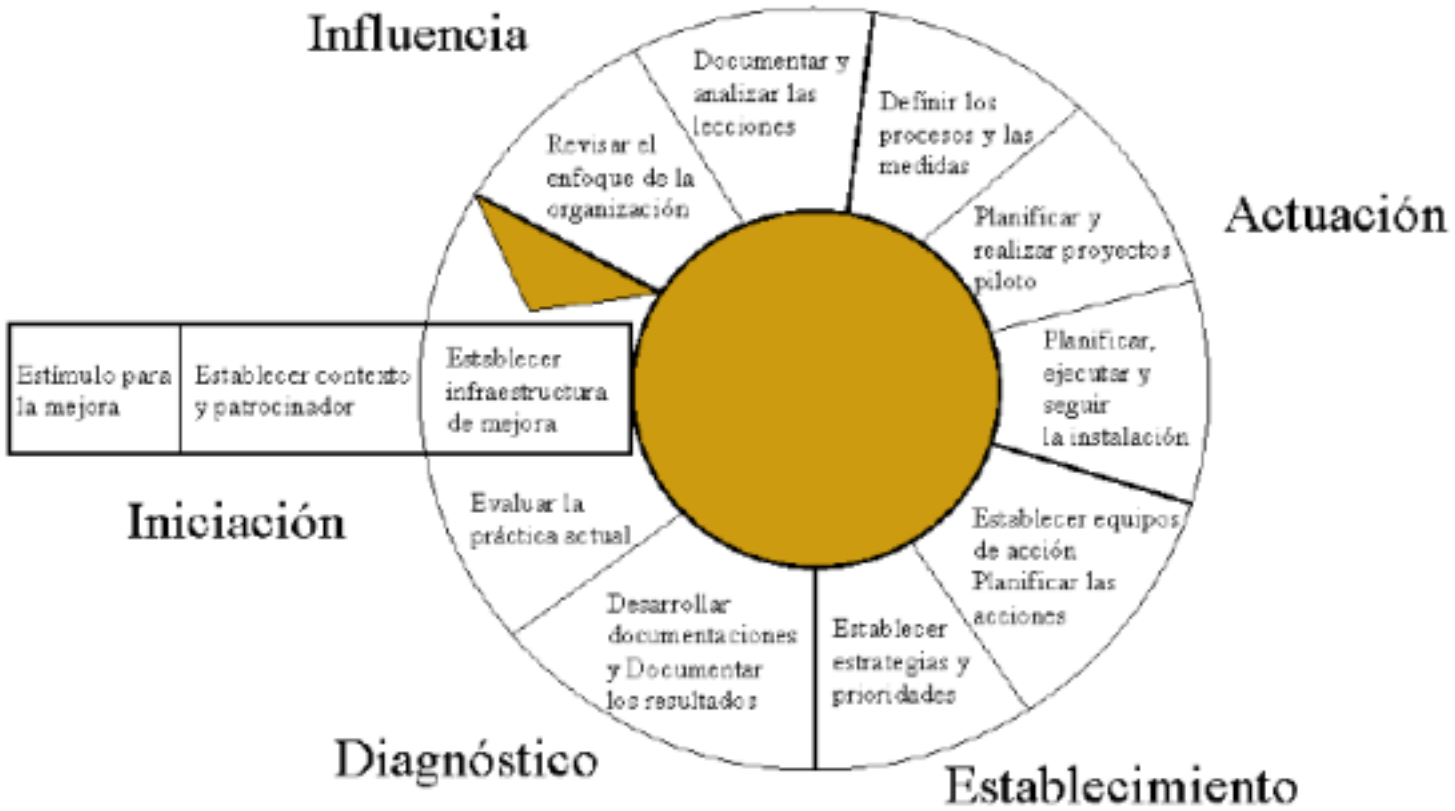
Métricas y Modelos de
Calidad de Software

Proceso de mejora del Software

- Mecanismo de mejora continua de la calidad que se basa en aplicar aquellas practicas que proporcionan buenos resultados y eliminar las que causan problemas.

Proceso de mejora del Software



Proceso de mejora del Software

- **Fiabilidad:** Probabilidad de operación libre de fallos de un software en un entorno determinado y durante un tiempo especifico. Tipos de Fallo:
  - Transitorio: Ocurre solo con ciertas entradas.
  - Permanente: Ocurre con todas las entradas.
  - Recuperable: El sistema se recupera sin intervención.
  - Irrecuperable: Se requiere intervención para recuperar el sistema.
  - No corruptor: No corrompe el estado del sistema o los datos.
  - Corruptor: Corrompe el estado del sistema o los datos.

- **Pruebas:** Contribuyen a mejorar la fiabilidad.

- **Revisiones:** Sirven para detectar los defectos que deban ser eliminados. Tipos de revisiones:
  - Revisiones de gestión: Controlan el progreso y detectan inconsistencias en la programación y requerimientos.
  - Revisiones técnicas: Revisar la documentación del proyecto.
  - Inspecciones: Revisiones que involucran al autor de un producto.
  - Walkthrough: Inspecciones por parte del grupo de desarrollo a partes especificas del producto.
  - Auditorias: Evaluaciones sobre el cumplimiento de estándares, planes, procedimientos, entre otros.

Modelos de Calidad de Software

- «El conjunto de características y las relaciones entre ellas que proveen la base para la especificación de los requisitos de calidad y la evaluación de la calidad» ISO/IEC 8402.

- Los modelos calidad permiten:
  - ✓ Definición estructurada de criterios de evaluación.
  - ✓ Especificación de requisitos con relación a ellos.
  - ✓ Descripción de componentes en un marco común.
  - ✓ Definición de métricas y prioridades.

Modelo McCall

- Organiza los factores en tres ejes o puntos de vista desde los cuales el usuario puede contemplar la calidad de un producto: Operación del Producto, Revisión del Producto, Transición del Producto.

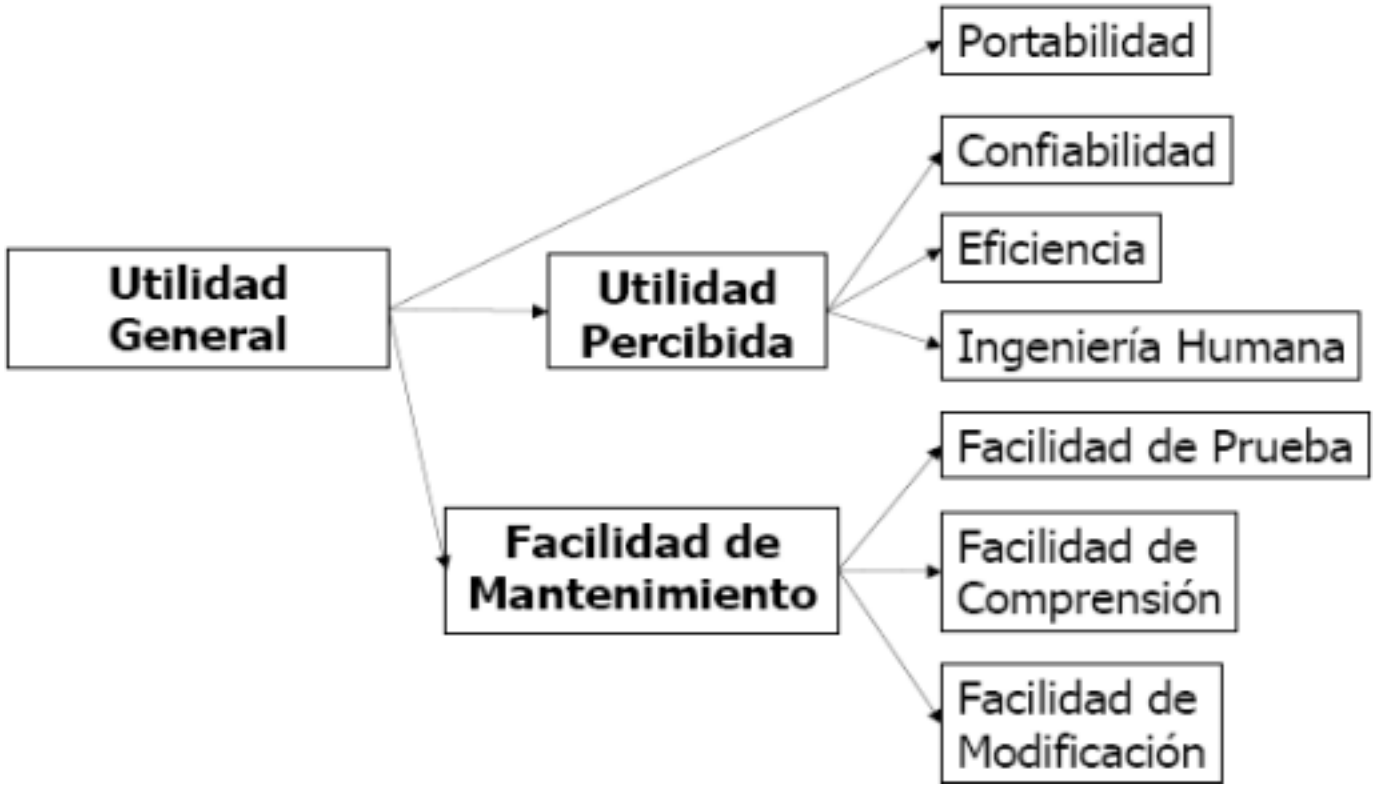
- **Operación del Producto:**
  - ✓ Facilidad de uso: ¿Puedo ejecutarlo?
  - ✓ Integridad: ¿Es seguro?
  - ✓ Corrección: ¿Hace el software lo que yo quiero?
  - ✓ Fiabilidad: ¿Lo hace de forma exacta?
  - ✓ Eficiencia: ¿Se ejecutara sobre el Hardware lo mejor posible?

- **Revisión del Producto:**
  - ✓ Facilidad de mantenimiento: ¿Puedo arreglarlo?
  - ✓ Facilidad de Prueba: ¿Puedo probarlo?
  - ✓ Flexibilidad: ¿Puedo modificarlo?

Modelo McCall

- **Transición del Producto:**
  ✓ Facilidad de reutilización: ¿Podre reutilizar parte del software? ✓ Interoperabilidad: ¿Podre comunicarlo con otros sistemas? ✓ Portabilidad: ¿Podre utilizarlo en otra maquina?

Modelo Boehm



Modelo ISO 9000

• Proporcionar una guía para la gestión de la calidad: diseño e implantación de sistemas de • calidad.

• ISO 9000 no normaliza el sistema de gestión de calidad, ya que esto depende del tipo de sector, tamaño de la empresa, organización interna, etc., sino que normaliza las verificaciones que se han de realizar sobre el sistema de calidad

• Describir los requerimientos generales para garantizar la calidad (demostrar la idoneidad del • sistema de calidad).

Modelo ISO 9000

• **ISO 9001:2000.** Modelo para conseguir la calidad total en el diseño, desarrollo, producción, instalación y servicio post-venta.
• **ISO 9000-3:1991.** Guía para la aplicación de la norma ISO 9001 al desarrollo, suministro y mantenimiento de software.
  - En todo caso, nos certificaríamos según ISO 9000-3.
  - No añade ni cambia los requerimientos de la ISO 90001. Los amplia y aclara. • **ISO 9004-1:1994.** Gestión de la calidad y elementos del sistema de calidad establecer el QA). • **ISO 8402:1994.** Gestión de la calidad y garantía de la calidad. Vocabulario. • **ISO 12207:1995.** Procesos del ciclo de vida del software.
• **ISO/IEC 9126:1991.** Características de la calidad de un producto software. • **ISO/IEC 12119:1995.** Productos software: evaluación y test.
• **ISO/IEC 14102:1995.** Guía para la evaluación y selección de herramientas CASE.

Modelo ISO 9126

- Es un modelo jerárquico con seis atributos principales que contribuyen a la

calidad.

Modelo ISO 25010

• Es un modelo jerárquico con seis atributos principales que contribuyen a la calidad. • Adecuación funcional.
• Eficiencia de desempeño.

- Compatibilidad.
- Usabilidad.
- Fiabilidad.
- Seguridad.
- Mantenibilidad.
- Portabilidad.

SPICE

- Mejoramiento de procesos de software.

- Evaluación y mejora de procesos software.

- Inicio del proyecto 1.993.
- Se halla en fase de Informe Técnico.
- Es aplicable a cualquier organización o empresa que quiera mejorar la capacidad de cualquiera de sus procesos de software.
- Se puede utilizar como herramienta de evaluación del estado de los procesos de software de la empresa.
- Es independiente de la organización, modelo del ciclo de vida, metodología y tecnología.13

MoProSoft

- Estandariza y mejora cómo trabajan las organizaciones que desarrollan y mantienen software. • Busca que las empresas (en especial PyMEs) gestionen mejor sus proyectos, eleven su capacidad y compitan con estándares internacionales.

- Contiene 3 categorías y 9 subprocesos:
  - ✓ **Alta Dirección (DIR)**
    - ➢ Gestión de Negocio: Define la razón de ser, objetivos estratégicos, políticas y la dirección para el resto de procesos de la empresa. Recibe retroalimentación desde Gerencia para la mejora continua.
  - ✓ **Gerencia (GER)**
    - ➢ Gestión de Procesos: Define el mapa de procesos, los mejora, fija indicadores y metas de desempeño.
    - ➢ Gestión de Proyectos: Asegura que los proyectos estén alineados al plan estratégico (alcance, costo, tiempo, riesgos).
    - ➢ Gestión de Recursos: Provee recursos y mantiene la Base de Conocimiento. Incluye tres subprocesos: Recursos humanos y ambiente de trabajo. Bienes, servicios e infraestructura. Conocimiento de la organización.

MoProSoft

  - ✓ **Operación (OPE)**
    - ➢ Administración de Proyectos Específicos: Planifica y controla cada proyecto concreto (cronograma, avances, cambios, calidad).

➢ Desarrollo y Mantenimiento de Software: Ejecuta el ciclo de vida: requisitos, análisis, diseño, construcción, integración y pruebas de productos nuevos o modificados.

Six Sigma

- Busca mejorar procesos reduciendo al mínimo los defectos o errores, mediante un enfoque basado en datos y análisis estadístico.

- Utiliza dos enfoques:

✓ DMAIC: Para mejorar procesos existentes.

- D (Definir). Identifica el problema, el alcance y los objetivos.
- M (Medir). Recopila datos y medir el problema actual.
- A (Analizar). Determina la causa raíz del problema.
- I (Mejorar). Implementa soluciones para eliminar la causa raíz.
- C (Controlar). Establece controles para mantener la mejora.

✓ DMADV: Para diseñar procesos nuevos con alta calidad desde cero.

- D (Definir). Define los objetivos del proyecto y los requisitos del cliente.
- M (Medir). Recopila datos y métricas para entender necesidades y expectativas.
- A (Analizar). Examina opciones y diseños para cumplir requisitos.
- D (Diseñar). Crea el diseño detallado del nuevo producto.
- V (Verificar). Prueba el diseño con prototipos para asegurar que cumple con las especificaciones y necesidades.

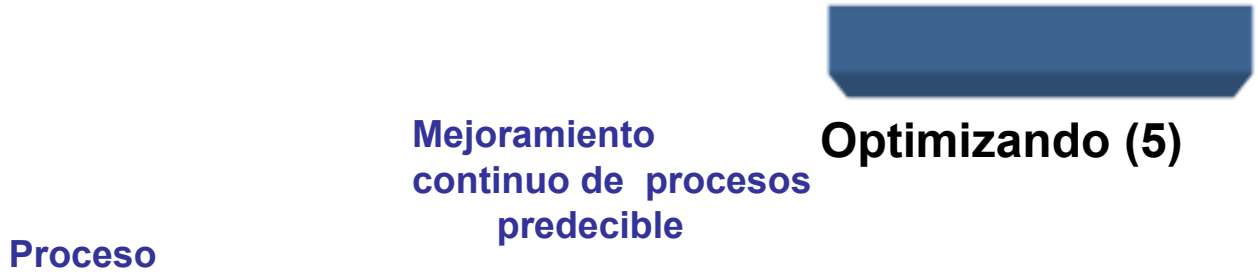
CMM

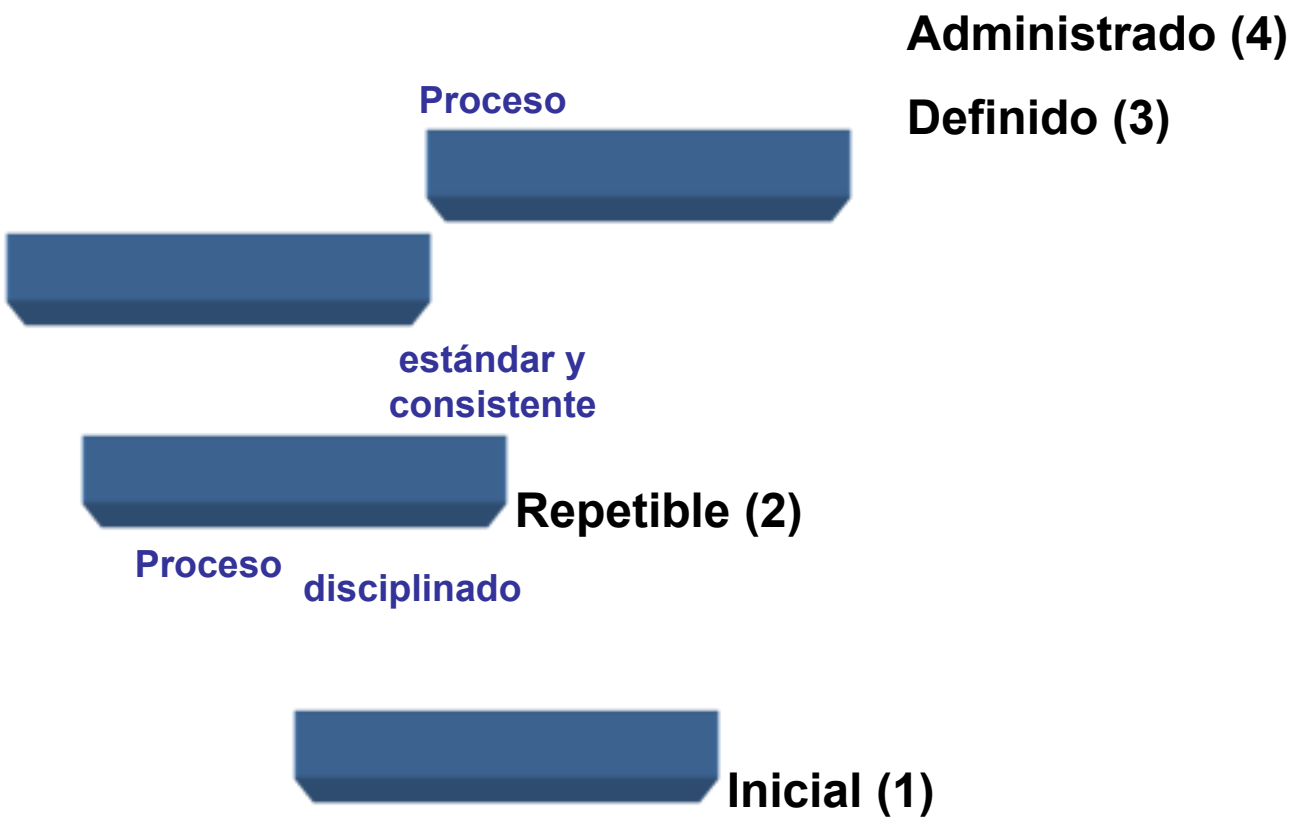
- Modelo de Calidad y Madurez.

- CMM es el máximo estándar en ingeniería de software Innovación, velocidad y satisfacción

del cliente se han convertido en la consigna de las organizaciones que quieren sobrevivir y crecer en el cada vez más competitivo mundo moderno. Como las tecnologías de información resultan fundamentales para lograrlas, el software se ha constituido en la piedra angular sobre la cual se soportan la gran mayoría de los nuevos modelos de empresa.

CMM





PSP

- Proceso Personal de Software.

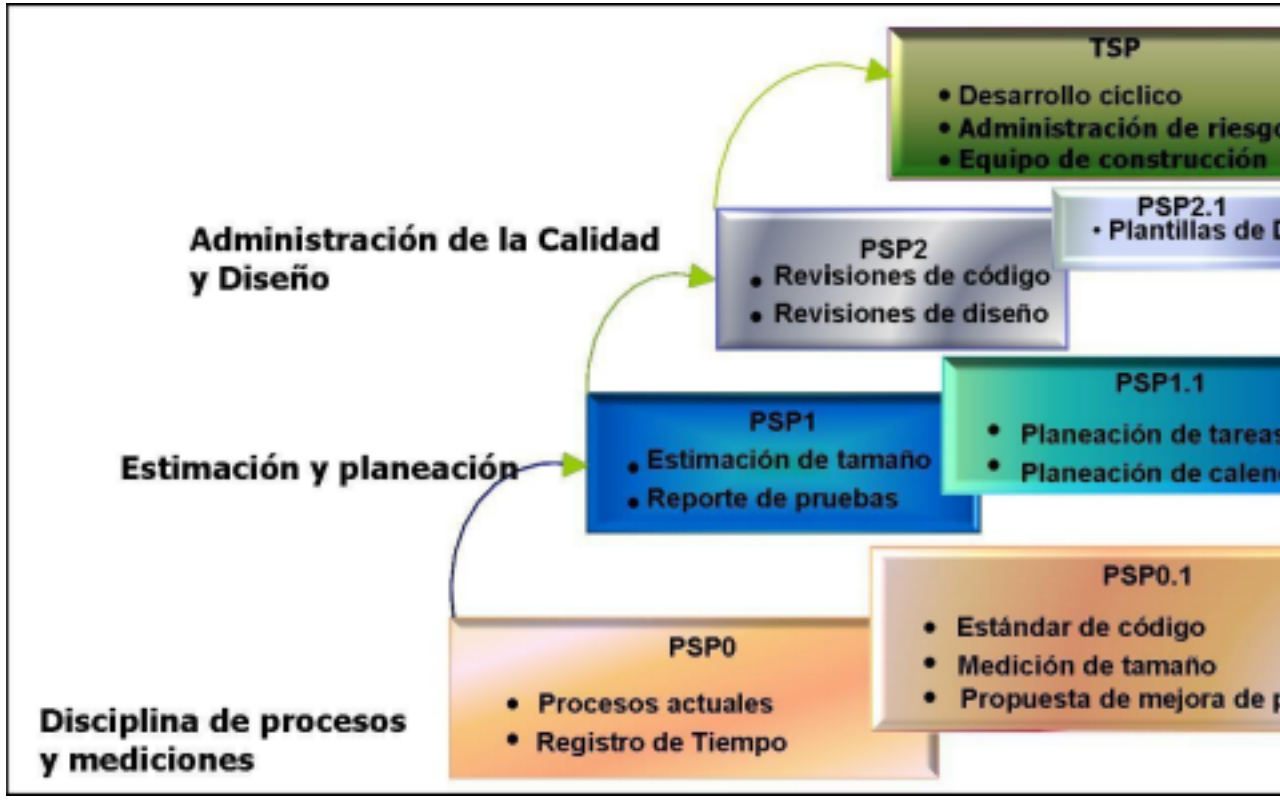
- Proceso de perfeccionamiento diseñado para controlar, gestionar y mejorar la forma de trabajar.

- Marco de trabajo estructurado con formularios, guías y procedimientos para desarrollar • software.

- Proporciona los datos históricos necesarios para mejorar procesos.

- Permite comprender por qué se han cometido errores y cuál es la mejor forma de encontrarlos.

PSP


TSP
• Desarrollo ciclico
• Administración de riesg
• Equipo de construcción
PSP2.1
• Plantillas de
Administración de la Calidad
y Diseño
PSP2
• Revisiones de código
• Revisiones de diseño
PSP1.1
• Planeación de tarea
• Planeación de calen
Estimación y planeación
PSP1
• Estimación de tamaño
• Reporte de pruebas
PSP0.1
• Estándar de código
• Medición de tamaño
• Propuesta de mejora de
PSP0
• Procesos actuales
• Registro de Tiempo
Disciplina de procesos
y mediciones

TSP

- Integración de Equipos de Desarrollo de Alto Rendimiento.

- Proporciona un marco de trabajo de procesos definidos que esta diseñado para ayudarle a equipos de gerentes e ingenieros a organizar y producir proyectos de software de gran escala.

- Mejora los niveles de calidad y productividad de un proyecto de desarrollo de software de un equipo, con el fin de ayudarlos a alcanzar los costos y tiempos en desarrollo.

TSP

EFQM

- Fundación Europea para la Gestión de la Calidad.

- El modelo EFQM surge en la década de los 80, en el ámbito de las empresas, ante la necesidad de ofrecer a los clientes, productos y servicios de mayor calidad, como única

forma de supervivencia ante la competencia internacional, procedentes sobre todo de Japón y Estados Unidos.

- Tiene como objetivo ayudar a las organizaciones (empresariales o de otros tipos) a conocerse mejor a sí mismas y, en consecuencia, a mejorar su funcionamiento.

EFQM

- Se basa en tres bloques principales:
  - ✓ **Dirección:** Evalúa cómo se hacen las cosas.
    - Liderazgo: Cómo los líderes inspiran, dirigen y apoyan el cambio.
    - Estrategia: Planes, visión y alineación de objetivos.
    - Personas: Gestión del talento y motivación.

- Alianzas y Recursos: Proveedores, tecnología, información.
- Procesos, Productos y Servicios: Cómo se diseña, gestiona y mejora lo que se hace. ✓ **Resultados:** Evalúa qué se logra.
- Resultados en las Personas: Satisfacción y compromiso interno.
- Resultados en Clientes: Satisfacción y fidelidad.
- Resultados en Sociedad: Impacto social y ambiental.
- Resultados Clave: Indicadores financieros y de desempeño.

✓ **Innovación y Aprendizaje:** Se aprende de la experiencia y la aplicación de la mejora continua.

Mapeo de Modelos

Resumen modelos de calidad

McCall	Evaluar el software desde la perspectiva del usuario y del desarrollador, considerando aspectos como usabilidad, eficiencia y mantenibilidad.	Para una aplicación de hoteles, medir la facilidad de uso, tiempo de respuesta y facilidad para corregir errores.
Boehm	Priorizar las características de calidad según necesidades del usuario y el contexto de uso.	Construir una aplicación para el sector de educación en el que se priorice la portabilidad y adaptabilidad a diferentes sistemas operativos.
ISO 9000:2015	Unificar conceptos y principios sobre calidad que sirvan de base para otros modelos.	Manual de calidad para una empresa de desarrollo que define términos y principios usados en todo el ciclo de vida.

Resumen modelos de calidad

ISO 9001:2015	Asegurar que los procesos cumplen estándares internacionales y mejoran continuamente.	Organizaciones que desarrollan ERP* y documentan procesos, controles y auditorías para certificación.
ISO/IEC 25000	Medir y mejorar la calidad del software entregado.	Medir en una aplicación de un banco los tiempos de respuesta, seguridad y accesibilidad.
CMMI	Mejorar procesos de desarrollo para alcanzar mayor previsibilidad y eficiencia.	Aplicación para el sector salud que pasa de nivel 2 a nivel 3 (Modelo de madurez) mejorando gestión de requisitos y control de cambios.

* ERP: Enterprise Resource Planning (Planificación de Recursos Empresariales). Es un software que integra y gestiona los procesos centrales de una empresa: contabilidad, ventas, inventario, compras, recursos humanos, etc.

Resumen modelos de calidad

MOPROSOFT	Estandarizar procesos de producción de software y gestión organizacional.	Pyme que desarrolla sistemas contables adopta MOPROSOFT para cumplir con estándares nacionales.
SPICE	Determinar el nivel de capacidad de los procesos y planificar mejoras.	Auditoría de calidad en una empresa que desarrolla aplicaciones para el sector de telecomunicaciones.
Six Sigma	Mejorar calidad reduciendo errores y optimizando procesos.	Equipo de QA* que reduce defectos en un sistema de facturación usando métricas DMAIC**.

* QA: Quality Assurance. Aseguramiento de la Calidad. ** DMAIC. Metodología de mejora continua.

Métricas de Calidad de Software

- Asignación de un valor a un atributo (tiempo, complejidad, entre otros), de un software, ya sea un producto (código) o un proceso (pruebas).
- Permiten dar respuesta a los siguientes interrogantes:
  - ¿Qué tan complejo es el código?
  - ¿Cuántos errores tiene?

  - ¿Qué tan mantenible es?

Clasificación

- **Métricas de Producto:** Evalúan las características del software.
  - ✓ Número de líneas de código (LOC).
  - ✓ Complejidad ciclomática (McCabe). Complejidad lógica del programa (cuántos condicionales, cuántos ciclos, cuántos for, etc).
  - ✓ Densidad de defectos.
  - ✓ Tiempo promedio de respuesta.

- **Métricas de Proceso:** Evalúan las actividades del desarrollo.
  - ✓ Tiempo promedio para el desarrollo de un módulo.
  - ✓ Porcentaje de pruebas realizadas.
  - ✓ Reingeniería o reestructuración por cambios en los requisitos.

- **Métricas del Proyecto:** Evalúan aspectos de gestión.
  - ✓ Costo estimado vs Costo real.
  - ✓ Cumplimiento del cronograma.
  - ✓ Número de tareas completadas-

Clasificación

- **Métricas basadas en los atributos internos del producto:** Evalúan las características estructurales del software que son visibles desde el código o la arquitectura. ✓ Complejidad ciclomática (McCabe).
  ✓ Acomplamiento.
  ✓ Cohesión.
  ✓ Herencia.

- **Métricas basadas en los atributos externos del producto:** Evalúan cómo se comporta el software en tiempo de ejecución, desde la perspectiva del usuario o entorno. ✓ Tiempos de respuesta.
  ✓ Tasa de fallos.
  ✓ Porcentaje de disponibilidad.
  ✓ Usabilidad percibida.

Clasificación

- **Métricas para Sistemas Orientados a Objetos:** Diseñadas para analizar sistemas que usan programación orientada a objetos.
  - WMC (Weighted Methods per Class): Complejidad total de los métodos en una clase. ○ DIT (Depth of Inheritance Tree): Profundidad de la jerarquía de herencia. ○ NOC (Number of Children): Número de subclases inmediatas de una clase. ○ CBO (Coupling Between Objects): Número de clases a las que está acoplada una clase. ○ RFC (Response For a Class): Número de métodos que pueden ejecutarse en respuesta a un mensaje recibido por un objeto.
  - LCOM (Lack of Cohesion of Methods): Mide qué tan relacionados están los métodos dentro de una clase.

Clasificación

- **Métricas Basadas en Código Fuente:** Evalúan el contenido y organización del código. ✓ LOC (Líneas de Código).
  ✓ Comentarios por línea de código.
  ✓ Complejidad ciclomática (McCabe).
  ✓ Profundidad de anidamiento.
  ✓ Número de funciones/métodos por clase.

- **Métricas Basadas en Estructura de Diseño:** Evalúan la arquitectura o modelo de diseño antes de la implementación.
  ✓ Número de módulos o componentes.
  ✓ Nivel de acoplamiento entre módulos.
  ✓ Grado de cohesión dentro de módulos.
  ✓ Trazabilidad entre requisitos y componentes.

En resumen

Atributos internos del producto Estructura del código.
Atributos externos del producto Comportamiento en ejecución.
Código fuente Calidad del código.
Estructura de diseño Arquitectura del sistema.
Sistemas orientados a objetos Diseño de la orientación a objetos,
Proceso Fases del desarrollo.
Proyecto Gestión y costos.

Ejemplo (Parte I Código Calculadora Simple)

```python
# Calculadora sencilla con operaciones básicas
#definimos la clase
class Calculadora:
    #creamos los metodos
    def __init__(self):
        pass

    def sumar(self, a, b):
        return a + b

    def restar(self, a, b):
        return a - b

    def multiplicar(self, a, b):
        return a * b

    def dividir(self, a, b):
        if b != 0:
            return a / b
        else:
            return None

#creamos el menú
def menu():
    print("1. Sumar")
    print("2. Restar")
    print("3. Multiplicar")
    print("4. Dividir")
    print("5. Salir")

def main():
    #instancia del objeto
    calc = Calculadora()
    opcion = 0
    while opcion != 5:
        menu()
        opcion = int(input("Seleccione una opción: "))
        if opcion in [1, 2, 3, 4]:
            x = float(input("Ingrese primer número: "))
            y = float(input("Ingrese segundo número: "))
            if opcion == 1:
                print("El resultado de la suma es: ",calc.sumar(x, y))
            elif opcion == 2:
                print("El resultado de la resta es: ", calc.restar(x, y))
            elif opcion == 3:
                print("El resultado de la multiplicación es: ", calc.multiplicar(x, y))
            elif opcion == 4:
                print("El resultado de la división es: ", calc.dividir(x, y))
        elif opcion == 5:
            print("Seleccionó la opcion Salir.")
        else:
            print("Opción inválida")

if __name__ == "__main__":
    main()
```

Ejemplo (Parte II Aplicación de la métrica)

Basada en código fuente	Total líneas: 55 Se incluyen comentarios y espacios.
	Líneas de código reales: 43 No se cuenta comentario ni espacios.
	Líneas de comentarios: 5
	Espacios en Blanco: 7
	*Densidad de comentarios: 11,6% Aceptable

* La Densidad de comentario se calcula con la siguiente fórmula:

$$\text{Densidad de comentarios} = \frac{\text{Líneas de comentarios}}{\text{Líneas de código reales}} * 100$$

Resultados:

✓ > 20 Muy bien documentado.
✓ >=10 and <=20 Aceptable.
✓ >= 5 and <10 Baja documentación.
✓ >=0 and < 5 Muy poca documentación

Ejemplo (Parte II Aplicación de la métrica)

Producto: Complejidad Ciclomática	Total condicionales clase Calculadora: 1 En el método dividir se tiene un if (else hace parte del if)
	Total condicionales y ciclos main: 7
	* CC = 8 + 1 = 9 * $\text{\textcircled{\textcircled{\textcircled{\textcircled{}}}}} = \text{\textcircled{\textcircled{\textcircled{\textcircled

Ejemplo (Parte II Aplicación de la métrica)

Orientadas a Objetos	WMC: 5 Total de métodos de la clase (init, sumar, restar, multiplicar, dividir)
	DIT: 1 Jerarquía de herencia (Herencia desde object)
	NOC: 0 Subclases
	CBO: 1 Bajo Acoplamiento * Número de clases a las que está acoplada una clase (solo se tiene el main desde allí se invoca la clase calculadora)
	RFC: 6 Bajo** métodos que pueden ejecutarse en respuesta a un mensaje recibido por un objeto (5 métodos propios sin llamar a métodos externos y 1 método externo)
	LCOM: Bajo *** Relación de métodos dentro de la clase (Todos los métodos operan sobre parámetros, no sobre atributos)

Ejemplo (Parte II Aplicación de la métrica)

* Para CBO, se tiene los siguientes niveles:
✓ 0 - 5 Bajo Acoplamiento. La clase usa pocas clases externas, alta independencia. Mantenible.
✓ 6 – 10 Media Acoplamiento. Dependencias moderadas, aceptable, pero se debe realizar supervisión. Tolerable. ✓ >10 Alto Acoplamiento. Muchas dependencias, difícil de mantener y probar. Revisión de diseño.

** Para RFC, se tiene la siguiente fórmula:

$$\text{RFC} = M + E$$

Donde:
M es el número de métodos que tiene la clase. En el ejercicio calculadora se tienen los métodos: init, sumar, restar, multiplicar y dividir. En total son 5.
E es el número de métodos externos distintos invocados por métodos de la clase. En el ejercicio calculadroa el método dividir llama a Except ValueError (constructor de excepción). En total es 1.

Y se tiene los siguientes niveles:
✓ 0 - 20 Bajo. Fácil de mantener y de probar.
✓ 21 – 50 Medio.
✓ >50 Alto. Más costoso de mantener.

*** Para LCOM, se tiene los siguientes niveles:
✓ 0 – 0,03 Alta Cohesión. Casi todos los métodos comparten atributos de instancia.
✓ 0,31 – 0,6 Media Cohesión. Algunos métodos comparten atributos otros no.

✓ >0,6 Baja Cohesión. Pocos o ningún atributo se comparte con otros métodos.

Ejemplo (Parte III Código Analizador Métricas)

Ejemplo (Parte III Código Analizador Métricas)

Ejemplo (Parte III Código Analizador Métricas)

Gracias